

TypeScript

Alunos: Antonio Victor, Diego, Gabriel de Jesus, João Vitor, Lucas Regis.

Critérios de avaliação:

- **Legibilidade:** A pergunta aqui é se conseguimos entender um código que não escrevemos sem recorrer à documentação. TypeScript ajuda: os tipos declarados mostram o que cada função recebe e devolve, e o compilador confere se isso continua correto, algo que um comentário não garante. O ponto fraco é que a linguagem mantém todo o comportamento do JavaScript, incluindo suas surpresas. Somar um número a um texto, por exemplo, ainda gera uma concatenação.
- **Redigibilidade:** Aqui importa quanto escrevemos antes da lógica. TypeScript reduz esse trabalho: a inferência dispensa a anotação de tipos na maioria dos casos, a equivalência estrutural permite passar um objeto a uma função sem declarar antes o seu tipo, e o código JavaScript existente pode ser migrado aos poucos. Em troca, dois tipos com a mesma forma são tratados como iguais. Para separá-los, como metros e pés, é preciso um artifício extra, enquanto linguagens nominais como Rust resolvem isso diretamente.
- **Confiabilidade:** A pergunta é quais erros a linguagem torna impossíveis e não apenas improváveis. Em TypeScript a resposta é moderada. Pesquisadores estimaram que a linguagem teria detectado cerca de 15% dos bugs reais analisados em projetos JavaScript, número considerado conservador, mas de uma versão antiga. Além disso, os tipos desaparecem na compilação, o “any” permite desligar a verificação e a documentação do projeto declara que um sistema de tipos sólido não é objetivo. O erro é antecipado no código tipado, mas os dados que vêm de fora continuam exigindo validação na execução.
- **Custo:** Custo vai além da velocidade. Na execução não há custo adicional, pois o JavaScript gerado não carrega os tipos, mas por isso também não há proteção nessa etapa. Na compilação, a versão 7.0 passou a usar um compilador nativo que, segundo a Microsoft, torna builds completos de 8 a 12 vezes mais rápidos (dado do fabricante, que não reproduzimos). Somam-se o custo de aprender e configurar a linguagem e o das falhas que os tipos não impedem.

Nomes e Escopos:

Em TypeScript, o modelo de nomes e escopos baseia-se na especificação do JavaScript, adicionando uma camada de tipagem avaliada estritamente em tempo de compilação.

Nomes e Vinculação:

- Os identificadores (nomes) referenciam dados em memória, funções e classes, mas também se vinculam a entidades exclusivas da linguagem que não existem na execução, como interface e type.
- A vinculação do **tipo** é estática e ocorre na compilação.
- A vinculação de **valor** e **endereço** continua dinâmica, ocorrendo no tempo de execução, com alocação em pilha (primitivos) ou heap implícita (objetos).

Modelo de Escopo:

O ambiente de referenciamento e as *closures* funcionam exatamente como descrito para JavaScript, retendo o acesso às variáveis de escopos superiores que já encerraram sua execução. O compilador determina a qual declaração se refere inspecionando a estrutura textual do código, subindo de blocos internos para externos até encontrar a definição.

- Declarações de Tipos: Variáveis de tipo (como o T em uma função genérica <T>) também possuem escopo léxico, mas seu ciclo de vida termina na compilação. Elas não ocupam endereço ou memória na execução devido ao processo de apagamento (*type erasure*)

Tempo de Vida de Variáveis e Closures:

O tempo de vida de uma variável refere-se ao período durante o qual ela permanece acessível na memória durante a execução de um programa. Esse tempo está diretamente relacionado ao conceito de escopo (scope), que define onde uma variável pode ser utilizada.

No TypeScript, existem três principais tipos de escopo:

- **Escopo global:** variáveis declaradas fora de funções ou blocos permanecem disponíveis durante toda a execução do programa.
- **Escopo de função:** variáveis declaradas dentro de uma função só existem enquanto essa função está sendo executada.
- **Escopo de bloco:** utilizando `let` e `const`, a variável existe apenas dentro do bloco delimitado por chaves `{}`.

De forma geral, uma variável deixa de existir quando o fluxo de execução sai do seu escopo. Nesse momento, ela se torna elegível para remoção da memória pelo mecanismo de coleta de lixo (garbage collection), desde que não haja mais referências a ela.

Extensão do Tempo de Vida / CLOSURE

Apesar da regra geral, há situações em que o tempo de vida de uma variável é estendido. Isso ocorre quando ainda existe alguma referência ativa a essa variável, impedindo que ela seja removida da memória. É nesse contexto que surge o conceito de closure. Uma closure pode ser definida como uma função que mantém acesso às variáveis do escopo no qual foi criada, mesmo após esse escopo ter sido finalizado. Isso ocorre porque, ao ser criada, a

função armazena uma referência ao seu ambiente léxico (lexical environment), que inclui as variáveis locais daquele contexto. Assim, mesmo que a função externa termine sua execução, as variáveis utilizadas pela função interna continuam acessíveis.

Aplicações Práticas

Closures são usadas principalmente para criar dados privados, manter estado entre execuções e em callbacks assíncronos que precisam lembrar do contexto original.

Vantagens e Desvantagens

Como vantagens, closures dão mais controle e segurança no acesso a dados, permitem reutilizar funções com estado independente e são base para padrões modernos. Como desvantagens, aumentam o consumo de memória, deixam o código mais complexo e podem causar erros difíceis de perceber.

Eixos de tipagem e de coerções:

No âmbito dos eixos de tipagem e de coerção, o TypeScript possui tipagem estática e detecta erros no código com base nos tipos, mesmo sem executá-lo. Além disso, durante a checagem, adota-se um julgamento próprio baseado nos tipos para avaliar a coerção implícita e exige-se que o programador especifique a conversão que o TS considerar adequada.

Dessa maneira, mesmo que a intenção original de um programador seja dividir um número por um array, o TypeScript é projetado para identificar o erro e dar ao programador a possibilidade de corrigi-lo. No entanto, como não é uma nova linguagem em si, e sim um *superset* do JavaScript, o TypeScript herda o *runtime behavior* do JavaScript, incluindo sua coerção implícita. Assim, o código ainda pode ser executado mesmo quando erros de tipo são encontrados durante a checagem, e conversões podem ocorrer automaticamente. Essas são características intencionais do TypeScript que facilitam a transição entre as duas linguagens. Dessa forma, mesmo com a mudança de JS para TS, é garantido que o código continuará a se comportar da mesma forma, ainda que o TS identifique erros.

Portanto, quando a checagem é concluída, o TypeScript apaga os tipos para produzir o código JS compilado "limpo". Nesse sentido, o código gerado não terá essas informações. Por isso, mesmo que apareçam erros de tipo durante a checagem, o TypeScript não mudará o *behavior* do código quando ele for executado. Logo, podemos concluir que o TypeScript, sob esse aspecto, adiciona a tipagem estática ao JavaScript, resultando em uma linguagem estrita na checagem, mas que mantém um runtime fraco por herdar a coerção implícita.

Equivalência e Inferência:

Equivalência de Tipos:

A equivalência de tipos determina quando dois tipos podem ser considerados iguais ou compatíveis. Existem principalmente duas abordagens:

- Equivalência nominal: dois tipos são equivalentes quando possuem o mesmo nome ou foram definidos pela mesma declaração.
- Equivalência estrutural: dois tipos são equivalentes quando possuem a mesma estrutura, independentemente de seus nomes.

Assim, a equivalência de tipos influencia a forma como uma linguagem verifica a compatibilidade entre valores e operações.

Inferência de Tipos:

A inferência de tipos permite que o compilador determine automaticamente o tipo de uma expressão ou variável a partir de seu valor e contexto, reduzindo a necessidade de declarações explícitas.

Em linguagens estaticamente tipadas, a inferência ocorre durante a compilação. O compilador pode gerar restrições sobre os tipos envolvidos e resolvê-las por meio de mecanismos como unificação.

É importante destacar que **inferência de tipos não significa tipagem dinâmica**. Na inferência, o tipo é determinado automaticamente, mas continua sendo verificado estaticamente antes da execução.

Equivalência e compatibilidade de tipos de TypeScript: TypeScript adota um sistema de tipos predominantemente estrutural, no qual a compatibilidade entre tipos é determinada pela estrutura de seus membros, e não exclusivamente pela identidade nominal de suas declarações. Um tipo-fonte pode ser atribuído a um tipo-alvo quando possui os membros necessários deste último em formas compatíveis, caracterizando uma relação de subtipagem estrutural. Essa regra é aplicada recursivamente aos membros e também participa da verificação de argumentos de funções. Entretanto, o sistema possui regras específicas para construções como funções, classes, enums, propriedades excedentes e parâmetros opcionais, além de permitir determinados comportamentos não completamente seguros em razão da compatibilidade com JavaScript.

Inferência de tipos de TypeScript: TypeScript realiza inferência estática para determinar tipos que não foram explicitamente anotados pelo programador. A inferência ocorre, entre outros contextos, na inicialização de variáveis, determinação de tipos de retorno, parâmetros contextualmente tipados, expressões e argumentos de tipos genéricos. O compilador pode calcular um melhor tipo comum para múltiplas expressões e utilizar tipagem contextual para determinar tipos a partir do contexto em que uma expressão aparece. A inferência não transforma TypeScript em uma linguagem dinamicamente tipada: os tipos inferidos continuam sendo utilizados durante a verificação estática antes da execução.